Министерство образования и науки Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

Факультет «Информатика и системы управления» (ИУ)

Кафедра «Информационные системы и телекоммуникации» (ИУ3)

Отчет

по лабораторной работе № 2

по курсу «Технологии машинного обучения»

Выполнил: Суржиков Л.А.

Группа: ИУ3С-51Б

Проверил: Ю.Е.Гапанюк

Москва, 2026 г.

1. Описание задания

Ссылка на постановку задания:

- https://github.com/ugapanyuk/courses_current/wiki/LAB_TMO__MISSING

Цель лабораторной работы:

изучение способов предварительной обработки данных для дальнейшего формирования моделей.

Задание:

- Выбрать набор данных (датасет), содержащий категориальные признаки и пропуски в данных. Для выполнения следующих пунктов можно использовать несколько различных наборов данных (один для обработки пропусков, другой для категориальных признаков и т.д.)
- Для выбранного датасета (датасетов) на основе материалов лекции решить следующие задачи: обработку пропусков в данных; кодирование категориальных признаков; масштабирование данных.

Требования к отчёту:

Отчёт по лабораторной работе должен содержать:

- титульный лист;
- описание задания;
- текст программы;
- экранные формы с примерами выполнения программы.

В случае использования ноутбуков фрагменты «текст программы» и «экранные формы» соответствуют ячейкам ноутбуков.

Выбранный датасет:

Loan Prediction Problem Dataset («Прогнозирование одобрения кредита», соревнование Analytics Vidhya, размещён на Kaggle) — 614 заявок клиентов жилищной финансовой компании на получение кредита: пол, семейное положение, число иждивенцев, образование, самозанятость, доход заявителя и созаёмщика, сумма и срок кредита, кредитная история, район недвижимости и целевой признак Loan_Status (кредит одобрен / не одобрен). В датасете 7 столбцов с пропусками и 7 категориальных признаков.

2. Текст программы и экранные формы с примерами выполнения

Ниже приведены фрагменты кода ноутбука LAB2_preprocessing_loan.ipynb и результаты их выполнения (текстовый вывод и графики). Фрагменты идут в порядке ячеек ноутбука.

Фрагмент 1 — Подключение библиотек

Код:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
import seaborn as sns

from sklearn.impute import SimpleImputer, MissingIndicator
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder, OneHotEncoder
from sklearn.preprocessing import MinMaxScaler, StandardScaler, Normalizer

matplotlib.rcParams["font.family"] = "DejaVu Sans"
matplotlib.rcParams["axes.unicode_minus"] = False

# палитра как в ЛР1: синий - исходные данные, красный - после обработки
BLUE_DARK = "#1B3B6F"
BLUE = "#3B6EA5"
BLUE_LIGHT = "#8FB8D9"
RED_DARK = "#7A1F2B"
RED = "#B23A48"
RED_LIGHT = "#E0949C"
```

Фрагмент 2 — 2. Описание набора данных

Код:

```
df = pd.read_csv("loan_prediction.csv")
df.head(10)
```

Результат выполнения:

```
    Loan_ID Gender Married Dependents     Education Self_Employed  \
0  LP001002   Male      No          0      Graduate            No
1  LP001003   Male     Yes          1      Graduate            No
2  LP001005   Male     Yes          0      Graduate           Yes
3  LP001006   Male     Yes          0  Not Graduate            No
4  LP001008   Male      No          0      Graduate            No
5  LP001011   Male     Yes          2      Graduate           Yes
6  LP001013   Male     Yes          0  Not Graduate            No
7  LP001014   Male     Yes         3+      Graduate            No
8  LP001018   Male     Yes          2      Graduate            No
9  LP001020   Male     Yes          1      Graduate            No

   ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0             5849                0.0         NaN             360.0
1             4583             1508.0       128.0             360.0
```

```
2             3000                0.0        66.0             360.0
3             2583             2358.0       120.0             360.0
4             6000                0.0       141.0             360.0
5             5417             4196.0       267.0             360.0
6             2333             1516.0        95.0             360.0
7             3036             2504.0       158.0             360.0
8             4006             1526.0       168.0             360.0
9            12841            10968.0       349.0             360.0

   Credit_History Property_Area Loan_Status
0             1.0         Urban           Y
1             1.0         Rural           N
2             1.0         Urban           Y
3             1.0         Urban           Y
4             1.0         Urban           Y
5             1.0         Urban           Y
6             1.0         Urban           Y
7             0.0     Semiurban           N
8             1.0         Urban           Y
9             1.0     Semiurban           N
```

Фрагмент 3 — 2. Описание набора данных

Код:

```
print("Размер датасета:", df.shape)
df.info()
```

Результат выполнения:

```
Размер датасета: (614, 13)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Loan_ID            614 non-null    object
 1   Gender             601 non-null    object
 2   Married            611 non-null    object
 3   Dependents         599 non-null    object
 4   Education          614 non-null    object
 5   Self_Employed      582 non-null    object
 6   ApplicantIncome    614 non-null    int64
 7   CoapplicantIncome  614 non-null    float64
 8   LoanAmount         592 non-null    float64
 9   Loan_Amount_Term   600 non-null    float64
 10  Credit_History     564 non-null    float64
 11  Property_Area      614 non-null    object
 12  Loan_Status        614 non-null    object
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
```

Фрагмент 4 — 3.1. Поиск пропусков

Код:

```
# количество и процент пропусков по каждому столбцу
missing = pd.DataFrame({
    "Пропусков": df.isnull().sum(),
```

```
    "Процент, %": (df.isnull().mean() * 100).round(2),
    "Тип": df.dtypes,
})
missing[missing["Пропусков"] > 0].sort_values("Пропусков", ascending=False)
```

Результат выполнения:

	Пропусков	Процент, %	Тип
Credit_History	50	8.14	float64
Self_Employed	32	5.21	object
LoanAmount	22	3.58	float64
Dependents	15	2.44	object
Loan_Amount_Term	14	2.28	float64
Gender	13	2.12	object
Married	3	0.49	object

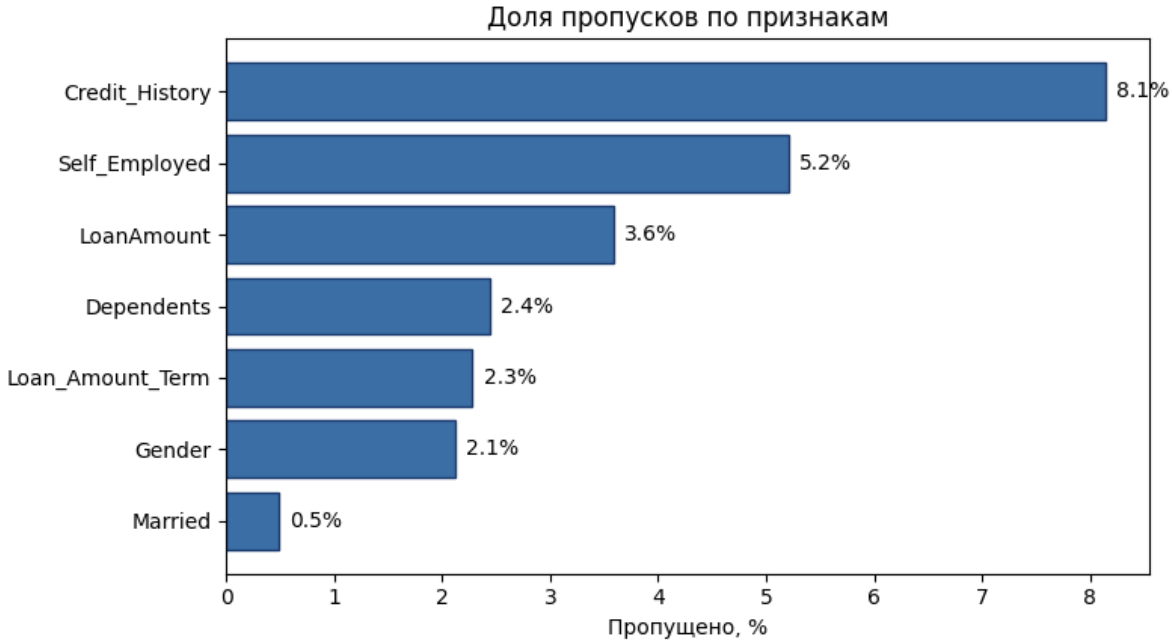
Фрагмент 5 — 3.1. Поиск пропусков

Код:

```
miss = df.isnull().mean().sort_values() * 100
miss = miss[miss > 0]

plt.figure(figsize=(8, 4.5))
plt.barh(miss.index, miss.values, color=BLUE, edgecolor=BLUE_DARK)
for i, v in enumerate(miss.values):
    plt.text(v + 0.1, i, f"{v:.1f}%", va="center")
plt.title("Доля пропусков по признакам")
plt.xlabel("Пропущено, %")
plt.tight_layout()
plt.show()
```

Результат выполнения:



Фрагмент 6 — 3.1. Поиск пропусков

Код:

```
# разделим столбцы с пропусками на числовые и категориальные
num_cols_na = [c for c in df.columns if df[c].isnull().sum() > 0 and df[c].dtype != "object"]
cat_cols_na = [c for c in df.columns if df[c].isnull().sum() > 0 and df[c].dtype == "object"]
print("Числовые с пропусками:", num_cols_na)
print("Категориальные с пропусками:", cat_cols_na)
```

Результат выполнения:

```
Числовые с пропусками: ['LoanAmount', 'Loan_Amount_Term', 'Credit_History']
Категориальные с пропусками: ['Gender', 'Married', 'Dependents', 'Self_Employed']
```

Фрагмент 7 — 3.2. Удаление строк и столбцов с пропусками

Код:

```
df_drop_rows = df.dropna(axis=0, how="any")
df_drop_cols = df.dropna(axis=1, how="any")

print("Исходный датасет:", df.shape)
print("Удалены строки с пропусками:", df_drop_rows.shape,
      f"- потеряно {df.shape[0] - df_drop_rows.shape[0]} строк "
      f"({(1 - df_drop_rows.shape[0] / df.shape[0]) * 100:.1f}%)")
print("Удалены столбцы с пропусками:", df_drop_cols.shape,
      f"- осталось столбцов: {list(df_drop_cols.columns)}")
```

Результат выполнения:

```
Исходный датасет: (614, 13)
Удалены строки с пропусками: (480, 13) - потеряно 134 строк (21.8%)
Удалены столбцы с пропусками: (614, 6) - осталось столбцов: ['Loan_ID', 'Education',
'ApplicantIncome', 'CoapplicantIncome', 'Property_Area', 'Loan_Status']
```

Фрагмент 8 — 3.3. Заполнение пропусков в числовых признаках

Код:

```
print("Среднее LoanAmount:", round(df["LoanAmount"].mean(), 2))
print("Медиана LoanAmount:", df["LoanAmount"].median())

loan_zero = df["LoanAmount"].fillna(0)
loan_mean = df["LoanAmount"].fillna(df["LoanAmount"].mean())
loan_median = df["LoanAmount"].fillna(df["LoanAmount"].median())

pd.DataFrame({
    "Исходный": df["LoanAmount"].describe(),
    "Нулями": loan_zero.describe(),
    "Средним": loan_mean.describe(),
    "Медианой": loan_median.describe(),
}).round(2)
```

Результат выполнения:

```
Среднее LoanAmount: 146.41
Медиана LoanAmount: 128.0

       Исходный  Нулями  Средним  Медианой
count    592.00  614.00   614.00    614.00
mean     146.41  141.17   146.41    145.75
std       85.59   88.34    84.04     84.11
```

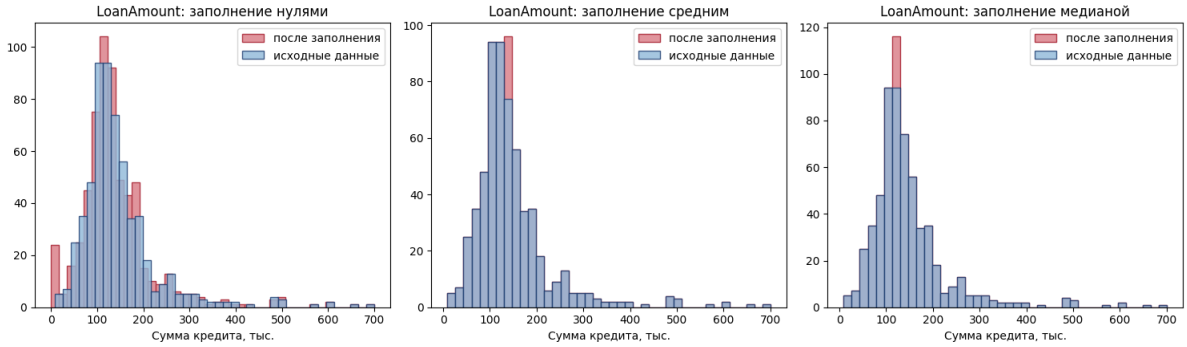
```
min        9.00    0.00     9.00      9.00
25%      100.00   98.00   100.25    100.25
50%      128.00  125.00   129.00    128.00
75%      168.00  164.75   164.75    164.75
max      700.00  700.00   700.00    700.00
```

Фрагмент 9 — 3.3. Заполнение пропусков в числовых признаках

Код:

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
variants = [("заполнение нулями", loan_zero), ("заполнение средним", loan_mean),
            ("заполнение медианой", loan_median)]
for ax, (name, s) in zip(axes, variants):
    ax.hist(s, bins=40, color=RED_LIGHT, edgecolor=RED, label="после заполнения")
    ax.hist(df["LoanAmount"].dropna(), bins=40, color=BLUE_LIGHT, edgecolor=BLUE_DARK,
            alpha=0.8, label="исходные данные")
    ax.set_title("LoanAmount: " + name)
    ax.set_xlabel("Сумма кредита, тыс.")
    ax.legend()
plt.tight_layout()
plt.show()
```

Результат выполнения:



Фрагмент 10 — 3.3. Заполнение пропусков в числовых признаках

Код:

```
print("Значения Loan_Amount_Term:")
print(df["Loan_Amount_Term"].value_counts(dropna=False).head(6))
print()
print("Значения Credit_History:")
print(df["Credit_History"].value_counts(dropna=False))
```

Результат выполнения:

```
Значения Loan_Amount_Term:
Loan_Amount_Term
360.0    512
180.0     44
480.0     15
NaN       14
300.0     13
240.0      4
Name: count, dtype: int64
```

```
Значения Credit_History:
Credit_History
1.0    475
0.0     89
NaN     50
Name: count, dtype: int64
```

Фрагмент 11 — 3.3. Заполнение пропусков в числовых признаках

Код:

```
# индикатор пропусков - запомним, в каких строках были пропуски
indicator = MissingIndicator()
mask = indicator.fit_transform(df[num_cols_na])
print("Пропуски по столбцам (из MissingIndicator):",
      dict(zip(num_cols_na, mask.sum(axis=0).tolist())))

df_filled = df.copy()

imp_median = SimpleImputer(strategy="median")
df_filled[["LoanAmount"]] = imp_median.fit_transform(df_filled[["LoanAmount"]])

imp_mode = SimpleImputer(strategy="most_frequent")
df_filled[["Loan_Amount_Term", "Credit_History"]] = imp_mode.fit_transform(
    df_filled[["Loan_Amount_Term", "Credit_History"]])

print("Медиана, которой заполнили LoanAmount:", imp_median.statistics_)
print("Мода для Loan_Amount_Term и Credit_History:", imp_mode.statistics_)
print()
print(df_filled[num_cols_na].isnull().sum())
```

Результат выполнения:

```
Пропуски по столбцам (из MissingIndicator): {'LoanAmount': 22, 'Loan_Amount_Term': 14,
'Credit_History': 50}
Медиана, которой заполнили LoanAmount: [128.]
Мода для Loan_Amount_Term и Credit_History: [360.   1.]

LoanAmount          0
Loan_Amount_Term    0
Credit_History      0
dtype: int64
```

Фрагмент 12 — 3.4. Заполнение пропусков в категориальных признаках

Код:

```
for col in cat_cols_na:
    print(col, df[col].value_counts(dropna=False).to_dict())
```

Результат выполнения:

```
Gender {'Male': 489, 'Female': 112, nan: 13}
Married {'Yes': 398, 'No': 213, nan: 3}
Dependents {'0': 345, '1': 102, '2': 101, '3+': 51, nan: 15}
Self_Employed {'No': 500, 'Yes': 82, nan: 32}
```

Фрагмент 13 — 3.4. Заполнение пропусков в категориальных признаках

Код:

```
# вариант 1 - отдельная категория "NA"
imp_const = SimpleImputer(strategy="constant", fill_value="NA")
self_emp_const = imp_const.fit_transform(df[["Self_Employed"]])
print("Self_Employed, заполнение константой:")
print(pd.Series(self_emp_const.ravel()).value_counts())
```

Результат выполнения:

```
Self_Employed, заполнение константой:
No     500
Yes     82
NA      32
Name: count, dtype: int64
```

Фрагмент 14 — 3.4. Заполнение пропусков в категориальных признаках

Код:

```
# вариант 2 - мода (его и оставляем: пропусков немного, до 5%)
imp_cat = SimpleImputer(strategy="most_frequent")
df_filled[cat_cols_na] = imp_cat.fit_transform(df_filled[cat_cols_na])

print("Мода по столбцам:", dict(zip(cat_cols_na, imp_cat.statistics_)))
print()
for col in cat_cols_na:
    print(col, df_filled[col].value_counts().to_dict())
```

Результат выполнения:

```
Мода по столбцам: {'Gender': 'Male', 'Married': 'Yes', 'Dependents': '0', 'Self_Employed': 'No'}

Gender {'Male': 502, 'Female': 112}
Married {'Yes': 401, 'No': 213}
Dependents {'0': 360, '1': 102, '2': 101, '3+': 51}
Self_Employed {'No': 532, 'Yes': 82}
```

Фрагмент 15 — 3.4. Заполнение пропусков в категориальных признаках

Код:

```
print("Пропусков осталось:", df_filled.isnull().sum().sum())
df_filled.head()
```

Результат выполнения:

```
Пропусков осталось: 0

    Loan_ID Gender Married Dependents     Education Self_Employed  \
0  LP001002   Male      No          0      Graduate            No
1  LP001003   Male     Yes          1      Graduate            No
2  LP001005   Male     Yes          0      Graduate           Yes
3  LP001006   Male     Yes          0  Not Graduate            No
4  LP001008   Male      No          0      Graduate            No

   ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0             5849                0.0       128.0             360.0
1             4583             1508.0       128.0             360.0
```

```
2             3000                0.0        66.0             360.0
3             2583             2358.0       120.0             360.0
4             6000                0.0       141.0             360.0

   Credit_History Property_Area Loan_Status
0             1.0         Urban           Y
1             1.0         Rural           N
2             1.0         Urban           Y
3             1.0         Urban           Y
4             1.0         Urban           Y
```

Фрагмент 16 — 4. Кодирование категориальных признаков

Код:

```
cat_cols = df_filled.select_dtypes(include="object").columns.drop("Loan_ID")
for col in cat_cols:
    print(f"{col:15} {df_filled[col].unique()}")
```

Результат выполнения:

```
Gender          ['Male' 'Female']
Married         ['No' 'Yes']
Dependents      ['0' '1' '2' '3+']
Education       ['Graduate' 'Not Graduate']
Self_Employed   ['No' 'Yes']
Property_Area   ['Urban' 'Rural' 'Semiurban']
Loan_Status     ['Y' 'N']
```

Фрагмент 17 — 4.1. Кодирование целочисленными значениями (LabelEncoder)

Код:

```
df_enc = df_filled.copy()
binary_cols = ["Gender", "Married", "Education", "Self_Employed", "Loan_Status"]

for col in binary_cols:
    le = LabelEncoder()
    df_enc[col] = le.fit_transform(df_enc[col])
    print(f"{col:15}", dict(zip(le.classes_, le.transform(le.classes_).tolist())))
```

Результат выполнения:

```
Gender          {'Female': 0, 'Male': 1}
Married         {'No': 0, 'Yes': 1}
Education       {'Graduate': 0, 'Not Graduate': 1}
Self_Employed   {'No': 0, 'Yes': 1}
Loan_Status     {'N': 0, 'Y': 1}
```

Фрагмент 18 — 4.2. Порядковое кодирование (OrdinalEncoder)

Код:

```
ord_enc = OrdinalEncoder(categories=[["0", "1", "2", "3+"]])
df_enc["Dependents"] = ord_enc.fit_transform(df_enc[["Dependents"]]).astype(int)

pd.concat([df_filled["Dependents"].rename("Dependents (было)"),
           df_enc["Dependents"].rename("Dependents (стало)")], axis=1).drop_duplicates()
```

Результат выполнения:

```
  Dependents (было)  Dependents (стало)
0                 0                   0
1                 1                   1
5                 2                   2
7                3+                   3
```

Фрагмент 19 — 4.3. One-hot кодирование (OneHotEncoder, get_dummies)

Код:

```
ohe = OneHotEncoder()
area_ohe = ohe.fit_transform(df_enc[["Property_Area"]])
print("Категории:", ohe.categories_)
print("Размер разреженной матрицы:", area_ohe.shape)
print(area_ohe.toarray()[:5])
```

Результат выполнения:

```
Категории: [array(['Rural', 'Semiurban', 'Urban'], dtype=object)]
Размер разреженной матрицы: (614, 3)
[[0. 0. 1.]
 [1. 0. 0.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]]
```

Фрагмент 20 — 4.3. One-hot кодирование (OneHotEncoder, get_dummies)

Код:

```
# то же самое через pandas, сразу получаем DataFrame с понятными названиями столбцов
area_dummies = pd.get_dummies(df_enc["Property_Area"], prefix="Area", dtype=int)
df_enc = pd.concat([df_enc.drop(columns=["Property_Area"]), area_dummies], axis=1)
df_enc.head()
```

Результат выполнения:

```
    Loan_ID  Gender  Married  Dependents  Education  Self_Employed  \
0  LP001002       1        0           0          0              0
1  LP001003       1        1           1          0              0
2  LP001005       1        1           0          0              1
3  LP001006       1        1           0          1              0
4  LP001008       1        0           0          0              0

   ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0             5849                0.0       128.0             360.0
1             4583             1508.0       128.0             360.0
2             3000                0.0        66.0             360.0
3             2583             2358.0       120.0             360.0
4             6000                0.0       141.0             360.0

   Credit_History  Loan_Status  Area_Rural  Area_Semiurban  Area_Urban
0             1.0            1           0               0           1
1             1.0            0           1               0           0
2             1.0            1           0               0           1
3             1.0            1           0               0           1
4             1.0            1           0               0           1
```

Фрагмент 21 — 4.3. One-hot кодирование (OneHotEncoder, get_dummies)

Код:

```
df_enc.info()
```

Результат выполнения:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 15 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   Loan_ID            614 non-null    object 
 1   Gender             614 non-null    int64  
 2   Married            614 non-null    int64  
 3   Dependents         614 non-null    int64  
 4   Education          614 non-null    int64  
 5   Self_Employed      614 non-null    int64  
 6   ApplicantIncome    614 non-null    int64  
 7   CoapplicantIncome  614 non-null    float64
 8   LoanAmount         614 non-null    float64
 9   Loan_Amount_Term   614 non-null    float64
 10  Credit_History     614 non-null    float64
 11  Loan_Status        614 non-null    int64  
 12  Area_Rural         614 non-null    int64  
 13  Area_Semiurban     614 non-null    int64  
 14  Area_Urban         614 non-null    int64  
dtypes: float64(4), int64(10), object(1)
memory usage: 72.1+ KB
```

Фрагмент 22 — 5. Масштабирование данных

Код:

```
scale_cols = ["ApplicantIncome", "CoapplicantIncome", "LoanAmount"]
df_enc[scale_cols].describe().round(2)
```

Результат выполнения:

```
       ApplicantIncome  CoapplicantIncome  LoanAmount
count           614.00             614.00      614.00
mean           5403.46            1621.25      145.75
std            6109.04            2926.25       84.11
min             150.00               0.00        9.00
25%            2877.50               0.00      100.25
50%            3812.50            1188.50      128.00
75%            5795.00            2297.25      164.75
max           81000.00           41667.00      700.00
```

Фрагмент 23 — 5.1. MinMax-масштабирование

Код:

```
mm = MinMaxScaler()
data_mm = pd.DataFrame(mm.fit_transform(df_enc[scale_cols]), columns=scale_cols)
data_mm.describe().round(3)
```

Результат выполнения:

```
       ApplicantIncome  CoapplicantIncome  LoanAmount
count          614.000            614.000     614.000
mean             0.065              0.039       0.198
std              0.076              0.070       0.122
min              0.000              0.000       0.000
25%              0.034              0.000       0.132
50%              0.045              0.029       0.172
75%              0.070              0.055       0.225
max              1.000              1.000       1.000
```

Фрагмент 24 — 5.2. Масштабирование на основе Z-оценки (StandardScaler)

Код:

```
ss = StandardScaler()
data_ss = pd.DataFrame(ss.fit_transform(df_enc[scale_cols]), columns=scale_cols)
data_ss.describe().round(3)
```

Результат выполнения:

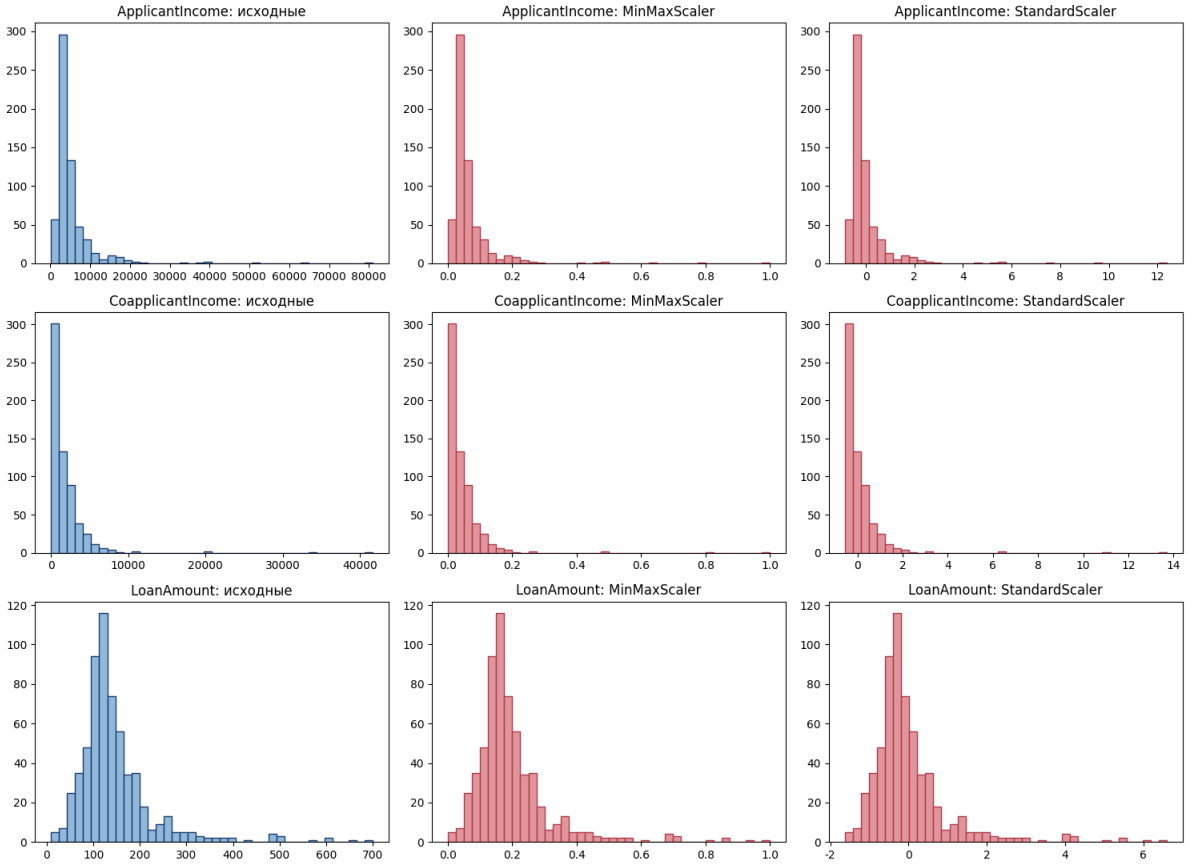
```
       ApplicantIncome  CoapplicantIncome  LoanAmount
count          614.000            614.000     614.000
mean            -0.000              0.000      -0.000
std              1.001              1.001       1.001
min             -0.861             -0.554      -1.627
25%             -0.414             -0.554      -0.541
50%             -0.261             -0.148      -0.211
75%              0.064              0.231       0.226
max             12.385             13.696       6.595
```

Фрагмент 25 — 5.2. Масштабирование на основе Z-оценки (StandardScaler)

Код:

```
fig, axes = plt.subplots(3, 3, figsize=(15, 11))
titles = ["исходные", "MinMaxScaler", "StandardScaler"]
colors = [(BLUE_LIGHT, BLUE_DARK), (RED_LIGHT, RED), (RED_LIGHT, RED)]
for i, col in enumerate(scale_cols):
    for j, data in enumerate([df_enc, data_mm, data_ss]):
        ax = axes[i, j]
        ax.hist(data[col], bins=40, color=colors[j][0], edgecolor=colors[j][1])
        ax.set_title(f"{col}: {titles[j]}")
plt.tight_layout()
plt.show()
```

Результат выполнения:



Фрагмент 26 — 5.3. Нормализация (Normalizer)

Код:

```
norm = Normalizer()
data_norm = pd.DataFrame(norm.fit_transform(df_enc[scale_cols]), columns=scale_cols)

print("Норма первых 5 строк после нормализации:")
print(np.linalg.norm(data_norm.head().values, axis=1).round(3))
data_norm.head()
```

Результат выполнения:

```
Норма первых 5 строк после нормализации:
[1. 1. 1. 1. 1.]

   ApplicantIncome  CoapplicantIncome  LoanAmount
0         0.999761           0.000000    0.021879
1         0.949565           0.312447    0.026521
2         0.999758           0.000000    0.021995
3         0.738107           0.673812    0.034291
4         0.999724           0.000000    0.023494
```

Фрагмент 27 — 5.4. Итоговый датасет

Код:

```
df_final = df_enc.copy()
df_final[scale_cols] = data_ss.values
df_final.head()
```

Результат выполнения:

```
    Loan_ID  Gender  Married  Dependents  Education  Self_Employed  \
0  LP001002       1        0           0          0              0
1  LP001003       1        1           1          0              0
2  LP001005       1        1           0          0              1
3  LP001006       1        1           0          1              0
4  LP001008       1        0           0          0              0

   ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0         0.072991          -0.554487   -0.211241             360.0
1        -0.134412          -0.038732   -0.211241             360.0
2        -0.393747          -0.554487   -0.948996             360.0
3        -0.462062           0.251980   -0.306435             360.0
4         0.097728          -0.554487   -0.056551             360.0

   Credit_History  Loan_Status  Area_Rural  Area_Semiurban  Area_Urban
0             1.0            1           0               0           1
1             1.0            0           1               0           0
2             1.0            1           0               0           1
3             1.0            1           0               0           1
4             1.0            1           0               0           1
```

Фрагмент 28 — 5.4. Итоговый датасет

Код:

```
print("Размер:", df_final.shape)
print("Пропусков:", df_final.isnull().sum().sum())
print("Нечисловые столбцы:", list(df_final.select_dtypes(exclude="number").columns))
```

Результат выполнения:

```
Размер: (614, 15)
Пропусков: 0
Нечисловые столбцы: ['Loan_ID']
```

3. Выводы

1. Выбран финансовый датасет заявок на кредит (614 записей, 13 признаков). Пропуски есть в 7 столбцах, больше всего — в Credit_History (8,1%).

2. Удалять строки с пропусками нецелесообразно: так теряется около 22% данных. Поэтому пропуски заполнили: LoanAmount — медианой, потому что распределение скошено; Loan_Amount_Term, Credit_History и категориальные признаки — модой.

3. Категориальные признаки закодированы: бинарные и целевой — через LabelEncoder, порядковый Dependents — через OrdinalEncoder, Property_Area — one-hot кодированием.

4. Денежные признаки отмасштабированы MinMaxScaler и StandardScaler, также показана построчная нормализация (Normalizer). Масштабирование не меняет форму распределения, а только переводит признаки в одну шкалу.

5. В итоге получен полностью числовой датасет без пропусков, который можно использовать для обучения моделей.